Brandon Nguyen, 827813045
COMPE596 - Spring 2026
3/16/2026

COMPE596 P06 - NVIDIA CUDA LU Factorization & Dense Linear Systems

1) Code Listing

// Main Program

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <cuda_runtime.h>
#include <cusolverDn.h>
#include <chrono>

int main(int argc, char*argv[])
{
  cusolverDnHandle_t cusolverH = NULL;
  cudaStream_t stream = NULL;

  cusolverStatus_t status = CUSOLVER_STATUS_SUCCESS;
  cudaError_t cudaStat1 = cudaSuccess;
  cudaError_t cudaStat2 = cudaSuccess;
  cudaError_t cudaStat3 = cudaSuccess;
  cudaError_t cudaStat4 = cudaSuccess;

  status = cusolverDnCreate(&cusolverH);
  assert(CUSOLVER_STATUS_SUCCESS == status);

  cudaStat1 = cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking);
  assert(cudaSuccess == cudaStat1);

  status = cusolverDnSetStream(cusolverH, stream);
  assert(CUSOLVER_STATUS_SUCCESS == status);


  for (int p = 0; p <= 10; p++) {

    int m = 1 << p;
```

```
int lda = m;
int ldb = m;

printf("\n===== N = %d =====\n", m);

double *A = (double*)malloc(sizeof(double)*m*m);
double *B = (double*)malloc(sizeof(double)*m);
double *B2 = (double*)malloc(sizeof(double)*m);
double *X = (double*)malloc(sizeof(double)*m);
double *LU = (double*)malloc(sizeof(double)*m*m);
int *Ipiv = (int*)malloc(sizeof(int)*m);

double *d_A = NULL;
double *d_B = NULL;
int *d_Ipiv = NULL;
int *d_info = NULL;
int lwork = 0;
double *d_work = NULL;
const int pivot_on = 0;

for (int j = 0; j < m; j++) {
  for (int i = 0; i < m; i++) {
    A[i + j*m] = 1.0 / (i + j + 1);
  }
}

for (int i = 0; i < m; i++) {
  B[i] = 1.0;
  B2[i] = 1.0 + ((double)rand() / RAND_MAX);
}

cudaStat1 = cudaMalloc ((void**)&d_A, sizeof(double) * lda * m);
cudaStat2 = cudaMalloc ((void**)&d_B, sizeof(double) * m);
cudaStat3 = cudaMalloc ((void**)&d_Ipiv, sizeof(int) * m);
cudaStat4 = cudaMalloc ((void**)&d_info, sizeof(int));
assert(cudaSuccess == cudaStat1);
assert(cudaSuccess == cudaStat2);
assert(cudaSuccess == cudaStat3);
assert(cudaSuccess == cudaStat4);
```

```
    cudaStat1 = cudaMemcpy(d_A, A, sizeof(double)*lda*m,
cudaMemcpyHostToDevice);
    cudaStat2 = cudaMemcpy(d_B, B, sizeof(double)*m, cudaMemcpyHostToDevice);
    assert(cudaSuccess == cudaStat1);
    assert(cudaSuccess == cudaStat2);

    status = cusolverDnDgetrf_bufferSize(
                                cusolverH,
                                m,
                                m,
                                d_A,
                                lda,
                                &lwork);
    assert(CUSOLVER_STATUS_SUCCESS == status);

    cudaStat1 = cudaMalloc((void**)&d_work, sizeof(double)*lwork);
    assert(cudaSuccess == cudaStat1);

    auto start1 = std::chrono::high_resolution_clock::now();


    if (pivot_on){
      status = cusolverDnDgetrf(
                            cusolverH,
                            m,
                            m,
                            d_A,
                            lda,
                            d_work,
                            d_Ipiv,
                            d_info);
    }else{
      status = cusolverDnDgetrf(
                            cusolverH,
                            m,
                            m,
                            d_A,
                            lda,
                            d_work,
                            NULL,
```

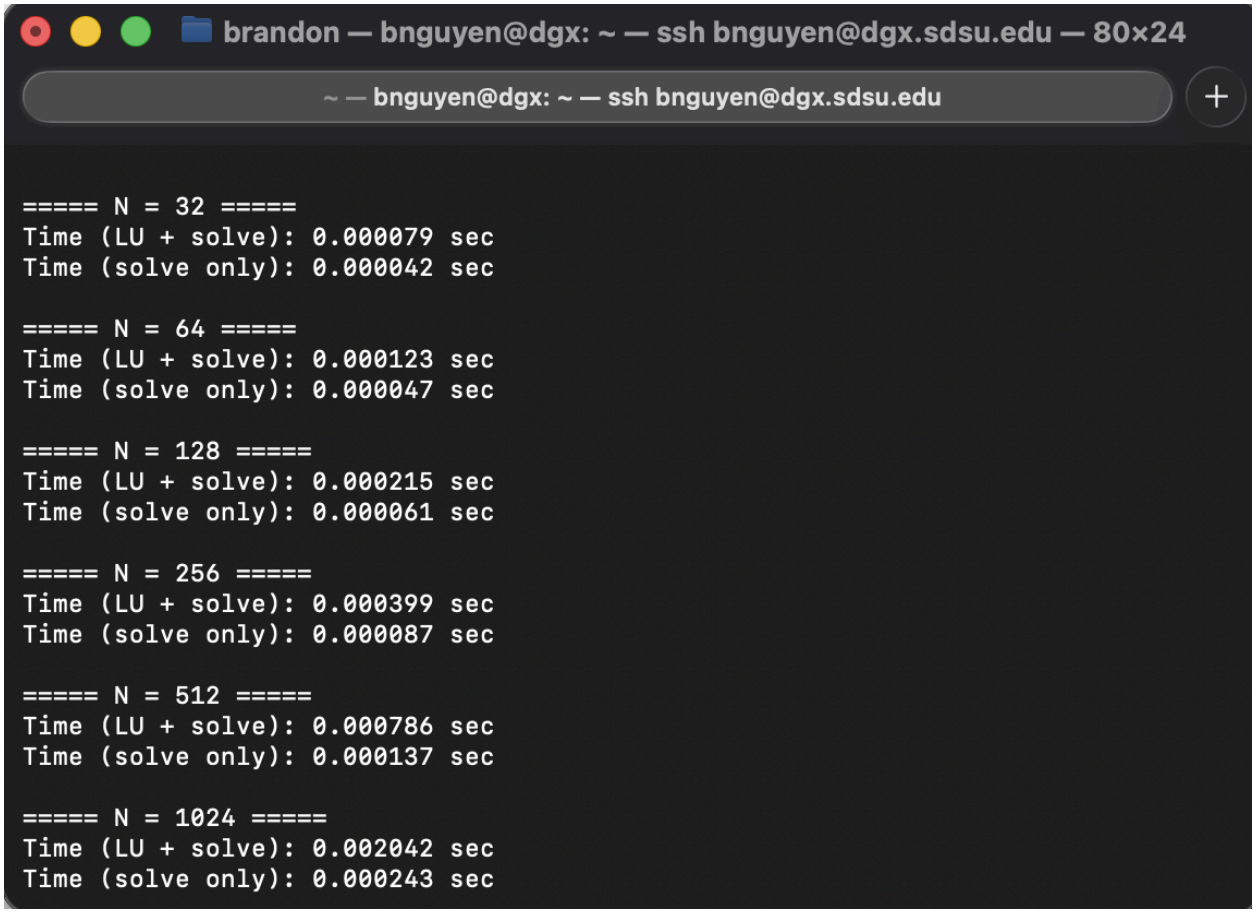
```
                        d_info);
}
cudaDeviceSynchronize();

if (pivot_on){
  status = cusolverDnDgetrs(
                        cusolverH,
                        CUBLAS_OP_N,
                        m,
                        1,
                        d_A,
                        lda,
                        d_Ipiv,
                        d_B,
                        ldb,
                        d_info);
}else{
  status = cusolverDnDgetrs(
                        cusolverH,
                        CUBLAS_OP_N,
                        m,
                        1,
                        d_A,
                        lda,
                        NULL,
                        d_B,
                        ldb,
                        d_info);
}

cudaDeviceSynchronize();
auto end1 = std::chrono::high_resolution_clock::now();

cudaMemcpy(X , d_B, sizeof(double)*m, cudaMemcpyDeviceToHost);
cudaMemcpy(d_B, B2, sizeof(double)*m, cudaMemcpyHostToDevice);

auto start2 = std::chrono::high_resolution_clock::now();

cusolverDnDgetrs(
                cusolverH,
```

```
                CUBLAS_OP_N,
                m,
                1,
                d_A,
                lda,
                NULL,
                d_B,
                ldb,
                d_info);

    cudaDeviceSynchronize();
    auto end2 = std::chrono::high_resolution_clock::now();
    double t1 = std::chrono::duration<double>(end1 - start1).count();
    double t2 = std::chrono::duration<double>(end2 - start2).count();

    printf("Time (LU + solve): %f sec\n", t1);
    printf("Time (solve only): %f sec\n", t2);

    free(A);
    free(B);
    free(B2);
    free(X);
    free(LU);
    free(Ipiv);

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_Ipiv);
    cudaFree(d_info);
    cudaFree(d_work);
  }

  cusolverDnDestroy(cusolverH);
  cudaStreamDestroy(stream);
  cudaDeviceReset();

  return 0;
}
```

// Makefile

```
NVCC = nvcc
TARGET = P06
SRC = P06.cu

LIBS = -lcusolver -lcudart

all: $(TARGET)

$(TARGET): $(SRC)
	$(NVCC) -o $(TARGET) $(SRC) $(LIBS)

run: $(TARGET)
	./$(TARGET)

clean:
	rm -f $(TARGET)
```

2) Program Output



3) Discussion

Question: Comment with one or two sentences explaining the value in performing LU factorization before solving multiple dense systems with the same coefficient matrix but different right-hand side vectors.

In P06, we are tasked with writing a program using the DGX server to observe GPU-accelerated solutions to dense linear systems using the NVIDIA cuSOLVER API. We will be writing a program to compare the elapsed time between solving the first system with LU factorization (cusolverDnDgetrf) and back substitution (cusolverDnDgetrs) also known as the **LU+solve method**, and solving the second system with the same LU factors determined using cusolverDnDgetrf from the first system, but with the second right-hand side using cusolverDnDgetrs, also known as the **solve only method**. According to the output terminal screenshot above, I used powers of two ranging from 2^1 to 2^10, otherwise known as 2 to 1024 for the N values. Here we notice that the elapsed time for both compared methods remains somewhat close for early powers of 2 or smaller values of N. However, as the value of N increases, the LU+solve method becomes slower and slower because of the more work it has to do. Therefore, to answer the question above and summarize the overall understanding, performing LU factorization first allows you to reuse the L and U factors for multiple right-hand sides, this makes solving many systems with the same coefficient matrix much faster overall. The solve-only method only performs forward and backward substitution, while the LU+solve has to do the same thing and then factorization as well, leading to a higher elapsed time. Taking everything into consideration, the sixth programming assignment furthered our understanding of CUDA programming by working with LU factorization and solving multiple dense systems.